

1. 题目深度解构

题意重述

对每个测试用例，给定全集 $\{1, 2, \dots, n\}$ 和一个大小为 k 的集合 S ，其中 $k < n / 2$ 。

第一次运行时，程序扮演 Alice。Alice 看到原集合 S ，需要往里面加入一个不属于 S 的数 x ，输出大小为 $k + 1$ 的集合。

第二次运行时，程序扮演 Bob。Bob 只看到 Alice 输出后的集合，也就是 $S \cup \{x\}$ ，需要判断 Alice 当时加入的到底是哪一个 x 。

关键限制是：第二次运行时程序会重启，测试用例顺序也可能打乱，所以不能依赖记忆、顺序、随机数、外部状态。Alice 和 Bob 必须使用同一套确定性规则。

数学上，本题要求我们构造一个映射：

$$f: \binom{[n]}{k} \rightarrow \binom{[n]}{k+1}$$

满足：

$$S \subset f(S)$$

并且 Bob 能从 $f(S)$ 唯一恢复出新增元素。

也就是说，我们需要在大小为 k 的所有子集和大小为 $k + 1$ 的所有子集之间，构造一个向上的单射匹配。

约束分析

题目给出：

$$n \leq 10^6, \quad T \leq 5 \times 10^5, \quad \sum n \leq 3 \times 10^6$$

因此每个测试用例允许 $O(n)$ ，整体 $O(\sum n)$ 可以接受。

不能预处理所有子集，因为组合数巨大。必须是一个按位扫描的显式构造。

特殊限制是：

$$k < \frac{n}{2}$$

这是突破口。它说明大小为 k 的层在布尔格中低于中间层，因此从第 k 层向第 $k + 1$ 层存在足够多的位置可以做匹配。

题目类型判定

这是一道组合构造题，核心是布尔格上的匹配构造。

更具体地说，它用到了 Greene-Kleitman symmetric chain decomposition，也可以理解为一种括号匹配构造。

2. 思维路径推导

最朴素的想法是：Alice 随便加一个不在集合里的数，例如最小的不在集合里的数。

但这会冲突。

例如 $n = 4, k = 1$ ：

- $S = \{1\}$ ，如果 Alice 加最小缺失数，得到 $\{1, 2\}$ ；
- $S = \{2\}$ ，如果 Alice 加最小缺失数，也得到 $\{1, 2\}$ 。

Bob 看到 $\{1, 2\}$ 时无法知道 Alice 原来加的是 1 还是 2。

所以问题本质不是“选一个缺失数”这么简单，而是要给每个 k 元集合分配一个唯一的 $(k+1)$ 元超集。

换句话说，我们需要构造一个匹配：

$$k\text{-subsets} \rightarrow (k+1)\text{-subsets}$$

因为 $k < n/2$ ，从组合数量上有：

$$\binom{n}{k} < \binom{n}{k+1}$$

数量是够的，但题目要求的是显式构造，而且 Bob 必须能反向识别。

关键观察是：把集合看成长度为 n 的 01 串。

如果第 i 个数在集合中，则第 i 位为 1，否则为 0。

然后做如下括号匹配：

- 把 0 看成左括号；
- 把 1 看成右括号；
- 从左到右扫描；
- 每遇到一个 1，就和它左边最近的尚未匹配的 0 配对。

也就是标准的栈匹配。

例如：

$$S = \{1, 3, 5\}, \quad n = 7$$

对应 01 串：

1 0 1 0 1 0 0

从左到右匹配：

- 位置 3 的 1 匹配位置 2 的 0；
- 位置 5 的 1 匹配位置 4 的 0；
- 位置 1 是未匹配的 1；
- 位置 6, 7 是未匹配的 0。

未匹配位置依次是：

1, 6, 7

其中前面是未匹配的 1，后面是未匹配的 0。

这个结构不是偶然的。所有未匹配位置从左到右一定形如：

若干个 1，然后若干个 0

原因是：如果某个未匹配的 0 出现在某个未匹配的 1 左边，那么扫描到那个 1 时，这个 0 应该还在栈中，于是它们会匹配，矛盾。

于是我们得到了一条链：

固定所有已匹配的位置，
只在未匹配位置上从左到右逐个把 0 变成 1。

例如未匹配位置是：

$c1 < c2 < c3 < c4$

那么同一条链上的集合形态是：

0000
1000
1100
1110
1111

放回原串中，匹配部分保持不变。

Alice 的操作就很自然了：

如果当前集合大小低于中间层，也就是 $k < n / 2$ ，那么未匹配的 0 一定存在。Alice 只需要把“最左边的未匹配 0”加入集合。

Bob 收到新集合后，用同样的括号匹配规则扫描。此时 Alice 新加的那个位置会变成“最右边的未匹配 1”。所以 Bob 删除并输出最右边的未匹配 1 即可。

这正好是一条 symmetric chain 上相邻两层之间的移动：

Alice: 向上走一步
Bob: 向下反推一步

正确性证明

设当前集合大小为 k 。

扫描匹配后，记：

- 匹配对数量为 p ；
- 未匹配的 1 数量为 u_1 ；
- 未匹配的 0 数量为 u_0 。

集合大小为：

$$k = p + u_1$$

非集合元素数量为：

$$n - k = p + u_0$$

两式相减：

$$u_0 - u_1 = n - 2k$$

因为题目保证：

$$k < \frac{n}{2}$$

所以：

$$n - 2k > 0$$

因此：

$$u_0 > u_1$$

所以一定存在未匹配的 0 。Alice 可以加入最左边的未匹配 0 。

加入后，匹配对不变，只是这个位置从未匹配的 0 变成未匹配的 1 。因此 Bob 对新集合重新做同样的匹配时，Alice 加入的位置正是最右边的未匹配 1 。

所以 Bob 输出最右边的未匹配 1 ，一定等于 Alice 加入的数。

3. Trick 与陷阱

核心 Trick 是括号匹配构造 symmetric chain decomposition。

这相当于把布尔格中的所有子集划分成若干条链，每条链从某个低层一直走到对称的高层。由于 $k < n / 2$ ，大小为 k 的集合在所在链中一定不是顶端，因此可以唯一地向上走一步。

易错点主要有这些：

第一，不能依赖测试用例顺序。第二轮 Bob 的测试用例可能被打乱，程序也会重启，所以必须是纯函数式的确定性策略。

第二，Bob 输入的是 $k + 1$ 个数，不是 k 个数。

第三，Alice 要加入的是“最左边的未匹配 0”，不是普通的最小缺失数。

第四，Bob 要输出的是“最右边的未匹配 1”。

第五，括号匹配方向不能反。这里是 0 匹配右侧的 1，也就是扫描时用栈保存未匹配的 0，遇到 1 就弹栈。

4. 代码实现

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    string s;
    cin >> s;

    int T;
    cin >> T;

    bool bob = s == "Bob";

    vector<char> in;
    vector<int> a, stk;

    while (T--) {
        int n, k;
        cin >> n >> k;

        int c = k + bob;

        in.assign(n, 0);
        a.resize(c);

        for (int i = 0; i < c; i++) {
            cin >> a[i];
            in[a[i] - 1] = 1;
        }

        stk.clear();
        if ((int) stk.capacity() < n) {
            stk.reserve(n);
        }

        int last = -1;

        for (int i = 0; i < n; i++) {
            if (!in[i]) {
```

```

        stk.push_back(i);
    } else {
        if (!stk.empty()) {
            stk.pop_back();
        } else {
            last = i;
        }
    }
}

if (!bob) {
    int x = stk[0] + 1;
    for (int i = 0; i < k; i++) {
        cout << a[i] << ' ';
    }
    cout << x << '\n';
} else {
    cout << last + 1 << '\n';
}

return 0;
}

```

时间复杂度：

$$O(n)$$

每个测试用例线性扫描一次。

总复杂度：

$$O\left(\sum n\right)$$

空间复杂度：

$$O(n)$$

5. 总结与启示

预估难度：Codeforces 2600 左右。

核心启示：当题目要求在相邻大小的子集层之间构造可逆映射时，应想到布尔格匹配，而 Greene-Kleitman 括号匹配提供了一个极其简洁的显式构造。

同类思维题可以练这些：

1. CSES — Gray Code
练习布尔超立方体上的显式构造。
2. AtCoder AGC031 C — Differ by 1 Bit
练习按位构造和超立方体路径。

3. Codeforces 1174D — Ehab and the Expected XOR Problem

练习利用二进制结构构造满足限制的集合序列。